D5 - Signatures & Authentication

Team 1
Gonzalez Fernandez Jonathan Uriel – 420051871 / jonathanuri123@gmail.com
Gabriel Hernández Yukioayax Canek – 321058526 / yukio.gabriel.h@gmail.com
Paz Rosas Jesús Eduardo – 315134854 / jesuspazrosas@gmail.com
Vázquez Apolonio Armando – 315140725 / armandovaz090@gmail.com
08 – Nov - 2026

1. Signature Design

Algorithm Chosen: The team selected Ed25519 (implemented via the cryptography library). Ed25519 is preferred over ECDSA for this use case because it is modern, secure, and fully deterministic. Unlike ECDSA, it does not rely on a fresh random nonce per signature, eliminating private-key recovery vulnerabilities caused by biased or compromised random number generators. It also produces compact 64-byte signatures and operates on 32-byte public and private keys. Furthermore, this selection enforces cryptographic domain separation across our architecture: secp256k1 is used exclusively for asymmetric DEK encapsulation (ECIES), while Ed25519 handles origin authentication, non-repudiation, and message integrity.

What Data is Signed: The signature covers a canonical JSON payload that binds:

1. **metadata:** Container version, original filename, AEAD algorithm, key wrap algorithm, and ISO 8601 UTC creation timestamp.
2. **recipients:** The complete, sorted list of authorized users, their key fingerprints, and their ECIES-encapsulated DEKs.
3. **nonce:** The 12-byte initialization vector used by AES-256-GCM.
4. **ciphertext:** The encrypted file payload (hex-encoded).
5. **tag:** The 16-byte AES-GCM GHASH authentication tag.
6. **signer_id:** The SHA-256 fingerprint of the sender's Ed25519 public key.

Why Hashing is Required Before Signing: Hashing maps a variable-length message into a fixed-size digest (a cryptographic fingerprint). Asymmetric signature primitives cannot efficiently or safely sign arbitrarily large payloads directly. In our architecture, the system performs an explicit SHA-256 pre-hashing step over the canonicalized JSON payload following the Hash -> Sign -> Verify model. This provides collision resistance, avalanche protection, and constant computational and memory overhead regardless of the container size.

2. Architecture & Container Schema (aad_builder & signature_manager)

In order to extend our system to guarantee origin authentication and non-repudiation, we updated the container format and validation engine:

Strict Schema Validation (validate_container & CAMPOS_CONTENEDOR): The container schema requires exactly seven fields: metadata, recipients, nonce, ciphertext, tag, signature, and signer_id. Any missing or unauthorized extra field triggers an immediate ContainerFormatError. We validate that the signature is a 64-byte lowercase hexadecimal string and signer_id is a 64-character hex string corresponding to a SHA-256 digest.

Canonical Payload Generation (build_signing_payload): To prevent JSON formatting discrepancies across operating systems and avoid injection attacks, the signed payload is serialized using strict canonical rules (sorted dictionary keys, ASCII encoding, and no whitespace delimiters: separators=(',', ':')). We also prepend a strict domain separation prefix (b"vault-d5-signature/v1\x00"). This guarantees that signature bytes cannot be replayed or reused in another cryptographic protocol.

Signer Identification (signer_id_from_ed25519_public_key): Instead of embedding the public key directly inside the container—where an adversary could replace it—we derive signer_id by computing the SHA-256 hash of the sender's raw 32-byte Ed25519 public key. The recipient uses this fingerprint to look up the sender's public key in an out-of-band trusted signers registry (trusted_signers).

3. Security Decisions

Why sign the ciphertext and not the plaintext? Signing the ciphertext (Encrypt-then-Sign) enforces a strict Fail-Closed architecture. The receiver can and must verify origin and integrity directly over the wire payload before invoking any decryption routine. If we signed the plaintext (Sign-then-Encrypt), an adversary who intercepts the container could re-encrypt the original plaintext under their own key and claim false ownership (Surrogate Sender Attack). Furthermore, the receiver would be forced to decrypt untrusted and potentially malicious bytes before being able to authenticate the sender.

What happens if the signature is not verified first? If signature verification is skipped or executed after decryption, the system becomes vulnerable to Denial of Service (DoS), side-channel attacks, and decryption oracles. Asymmetric DEK decapsulation (ECIES point multiplication) is computationally intensive; skipping initial verification allows attackers to exhaust CPU resources with crafted payloads. Verifying the signature as the first gate ensures forged or tampered containers are rejected immediately with an IntegrityError before allocating memory for decryption.

What happens if metadata is excluded from the signature? Excluding metadata enables Context Manipulation Attacks. An attacker could preserve the valid ciphertext and signature while altering the filename (e.g., swapping a safe extension for an executable), modifying the creation timestamp, or tampering with the recipients table. By binding both metadata and the recipients list directly into the signed payload, any structural tampering invalidates the signature.

4. Verification Workflow

Emission Pipeline

1. Generate an ephemeral 256-bit DEK using the OS CSPRNG (os.urandom(32)).
2. Encapsulate the DEK for each recipient using ECIES over secp256k1.
3. Encrypt the file bytes with AES-256-GCM using canonical metadata as AAD.
4. Build the canonical signing payload (metadata, recipients, nonce, ciphertext, tag, signer_id).
5. Compute the SHA-256 hash and sign it with the sender's Ed25519 private key.
6. Export the final validated D5 JSON container.

Reception Pipeline (Fail-Closed)

1. Parse the JSON container and validate structural schema compliance.
2. Extract signer_id and resolve the corresponding public key from the out-of-band trusted_signers registry.
3. Reconstruct the canonical signing payload and compute its SHA-256 hash.
4. Cryptographically verify the Ed25519 signature. If invalid or untrusted, raise an IntegrityError immediately and terminate execution.
5. Locate the recipient entry, unwrap the DEK using the recipient's secp256k1 private key via ECIES, and decrypt the authenticated payload using AES-256-GCM.

5. Acceptance Test Suite Verification

The acceptance test suite (tests/test_d5_acceptance.py) executes end-to-end integration tests using encrypt_hybrid() and decrypt_hybrid(), covering all mandatory scenarios:

Scenario A: Valid signature -> File accepted

- **Test:** test_scenario_a_valid_signature_accepted
- **Condition:** Authentic container signed by Alice, decrypted by Bob and Alice against trusted_signers.
- **Result:** PASS (100% bit-level fidelity restored).

Scenario B: Modified ciphertext -> Rejected

- **Test:** test_scenario_b_modified_ciphertext_rejected
- **Condition:** Inversion of a single nibble in ciphertext.
- **Result:** FAIL-CLOSED (Raised IntegrityError prior to DEK decapsulation).

Scenario C: Modified metadata -> Rejected

- **Test:** test_scenario_c_modified_metadata_rejected
- **Condition:** Tampered filename inside metadata dictionary.
- **Result:** FAIL-CLOSED (Raised IntegrityError due to signature mismatch).

Scenario D: Wrong public key -> Rejected

- **Test:** test_scenario_d_wrong_public_key_rejected
- **Condition:** Valid signature evaluated against an incorrect public key, and container signed by an unknown emisor.
- **Result:** FAIL-CLOSED (Raised IntegrityError; both cases blocked).

Scenario E: Signature removed -> Rejected

- **Test:** test_scenario_e_signature_removed_rejected
- **Condition:** Container stripped of signature or signer_id fields.
- **Result:** FAIL-CLOSED (Raised IntegrityError due to schema rejection).

Integration Roundtrip: JSON serialization

- **Test:** test_json_serialization_roundtrip
- **Condition:** Export to JSON string via json.dumps and reload via json.loads.
- **Result:** PASS (Container remains canonical and decryptable).

Fail-Closed Guard: Invalid signature bytes

- **Test:** test_invalid_signature_bytes_fails_closed
- **Condition:** Signature payload corrupted while maintaining length.

Result: FAIL-CLOSED (Raised IntegrityError without attempting decryption).

6. Conclusion

The integration of Ed25519 digital signatures with canonical signing payloads bridges the gap between symmetric data encryption (confidentiality) and non-repudiation (origin authentication and integrity). By implementing domain separation, strict schema validation in aad_builder.py, and a pre-decryption verification gate in hybrid_vault.py, the system guarantees that any tampered, forged, or unauthenticated file is rejected immediately without risking information leakage or unauthorized decapsulation.